

FORMATION CLOUD-NATIVE

Développement Cloud-Native avec AWS, C# et .NET

Plongez dans l'écosystème du développement moderne : architectures serverless, SDK AWS pour .NET et bonnes pratiques cloud. Une initiation pratique pour construire des applications scalables, résilientes et économiques.

Architecture d'une Web App Moderne

L'évolution des architectures logicielles est au cœur de la révolution cloud. Comprendre cette transition, c'est comprendre pourquoi le cloud-native existe.



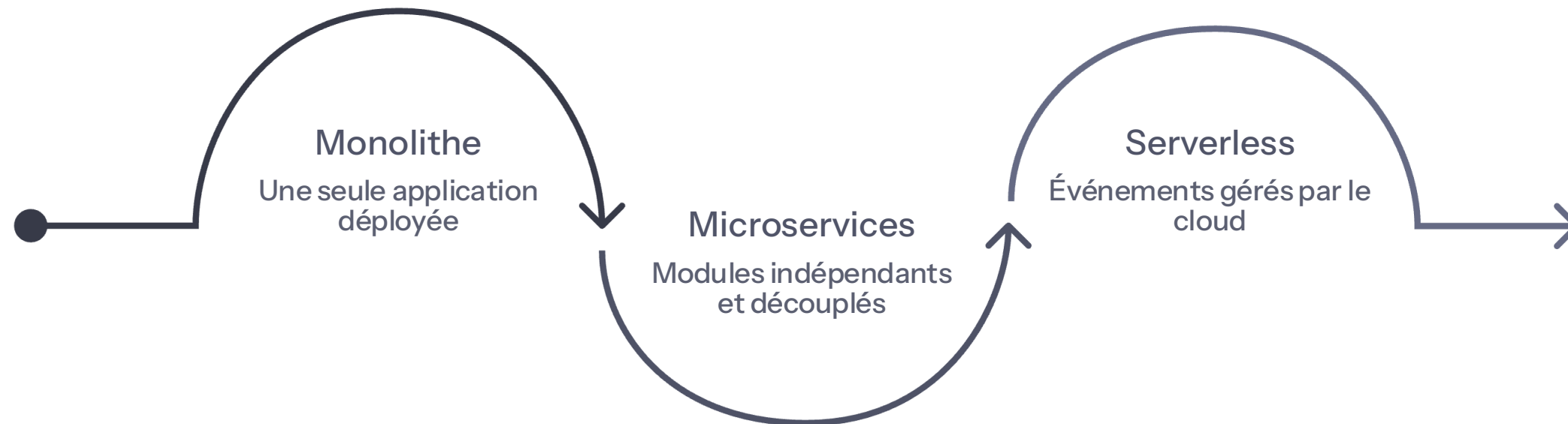
Monolithe

Un seul bloc de code déployé en une fois. Simple au départ, mais difficile à faire évoluer et à mettre à l'échelle.



Microservices

Des composants indépendants, déployables séparément. Agilité, scalabilité et réduction des coûts grâce au serverless.



Le serverless pousse la logique microservices à l'extrême : vous ne gérez plus aucun serveur, AWS s'occupe de l'infrastructure.

Accès aux services AWS

Trois portes d'entrée vers AWS mais pour un développeur, la console n'est qu'un outil de vérification. **Le code est l'instrument d'action.**

Console Web

Interface graphique pour explorer, vérifier et déboguer vos ressources.

AWS CLI

Ligne de commande pour automatiser les tâches et scripter vos déploiements.

SDK C# / .NET

Bibliothèques NuGet pour piloter AWS directement depuis votre code C#.

IAM – Autorisation

AWS Identity and Access Management contrôle **qui peut faire quoi** sur vos ressources. Règle d'or : le **principe du moindre privilège**.

Utilisateurs

Identités individuelles
avec des credentials
uniques.

Groupes

Regroupements logiques
pour appliquer des
politiques communes.

Rôles

Identités temporaires
pour services et
applications.

Politiques (JSON)

Documents déclaratifs
décrivant les autorisations
précises.

Configuration de l'environnement

Avant de coder, il faut préparer vos outils. Voici les trois étapes essentielles pour être opérationnel.

1 AWS Cloud9 ou IDE local

Environnement de développement intégré, prêt à l'emploi dans le navigateur ou via Visual Studio / Rider.

2 SDK AWS pour .NET

Installez les packages NuGet `AWSSDK.Core` et les clients spécifiques (S3, DynamoDB...).

3 Profil CLI

Exécutez `aws configure` pour enregistrer votre Access Key, Secret Key et région par défaut.

Amazon S3 – Concepts clés

S3 n'est pas un système de fichiers classique. C'est un **stockage d'objets HTTP** conçu pour la durabilité et l'échelle planétaire.



Buckets

Conteneurs de niveau supérieur avec un nom unique au monde.



Objets

Toute donnée stockée : fichier, image, backup.
Jusqu'à 5 To par objet.



Clés

Identifiant unique de l'objet dans le bucket (chemin logique).



Métadonnées

Paires clé-valeur décrivant l'objet (Content-Type, tags personnalisés...).

SDK C# & S3 — Upload par code

Apprenez à envoyer un fichier vers S3 en quelques lignes de C#. L'objet `AmazonS3Client` est votre point d'entrée.

```
using Amazon.S3;
using Amazon.S3.Model;

var client = new AmazonS3Client();

var request = new PutObjectRequest
{
    BucketName = "mon-bucket",
    Key       = "docs/rapport.pdf",
    FilePath  = "./rapport.pdf"
};

var response = await client
    .PutObjectAsync(request);

Console.WriteLine(
    $"Upload OK – HTTP {response.HttpStatusCode}");
```

AmazonS3Client

Client HTTP gérant l'authentification et la sérialisation.

PutObjectRequest

Décrit la destination (bucket + clé) et la source du fichier.

Async / Await

Toutes les opérations SDK sont asynchrones par défaut.

Introduction au NoSQL avec DynamoDB

DynamoDB offre une **flexibilité de schéma** et des performances à la milliseconde, sans gestion de serveur.

Concept	SQL (Relationnel)	DynamoDB (NoSQL)
Conteneur	Table avec schéma fixe	Table avec schéma flexible
Enregistrement	Row (ligne)	Item (document)
Champ	Column (colonne)	Attribute (attribut libre)
Requêtes	SQL / Jointures	Clé primaire / Scans

DYNAMODB

MODÉLISATION

Clés et Index dans DynamoDB

La modélisation des clés détermine la **performance de vos lectures**. Choisissez-les avant d'écrire la première ligne de code.

Partition Key (PK)

Distribue les données sur les nœuds de stockage. Doit être hautement cardinalitaire.

Sort Key (SK)

Trie les items au sein d'une partition. Permet les requêtes par plage (begins_with, between...).

GSI — Global Secondary Index

Nouvel axe de requête avec une PK et SK différentes. Capacité indépendante.

LSI — Local Secondary Index

Même Partition Key, Sort Key alternative. Défini uniquement à la création.

Le Modèle Objet C# — Object Persistence

Mappez une classe C# à une table DynamoDB, exactement comme Entity Framework le fait pour SQL. Les attributs .NET font le lien.

```
[DynamoDBTable("Produits")]
public class Produit
{
    [DynamoDBHashKey]
    public string Id { get; set; }

    [DynamoDBRangeKey]
    public string Categorie { get; set; }

    [DynamoDBProperty("nom")]
    public string Nom { get; set; }

    public decimal Prix { get; set; }
}
```

1

Classe C#

Attributs de mapping sur les propriétés

2

DynamoDBContext

ORM léger : SaveAsync, LoadAsync,
QueryAsync

3

Table DynamoDB

Items persistés automatiquement